📄 The Ultimate Master Doc: Design Patterns

1️⃣ Concept First: What is a Design Pattern?

The Problem:
When building large software, engineers kept running into the exact same problems over and over again (e.g., *"How do I make sure two parts of my app are using the same database connection?"*, *"How do I notify users when data changes?"*).

The Logic:
Instead of reinventing the wheel every time, experienced developers created standard, reusable templates to solve these common problems.

What it is NOT:
A design pattern is **not** actual code you can copy-paste.
It is a conceptual blueprint or a strategy for how to structure your classes and objects.

Why use them? (Exam Points)

Reusable Solutions:
Proven templates that save time.

Communication:
Developers share a common vocabulary (saying *"Let's use a Singleton"* instead of explaining a 10-minute concept).

Encapsulate Change:
They protect your core code when new features are added.

2️⃣ The 3 Main Categories

Think of patterns like sorting tools in a toolbox:

1. **Creational Patterns:** Deal with how objects are *created* (born).
2. **Structural Patterns:** Deal with how objects are *connected* or assembled into larger structures.
3. **Behavioral Patterns:** Deal with how objects *communicate* and behave with each other.

3️⃣ Theory Understanding (The Core Patterns)

A. Singleton Pattern (Creational)

Meaning:
Ensures that a class has **only one instance** (one object) running in the entire system, and provides a global point to access it.

ClinixTech Example:
The **Database Connection**. You do not want 50 different doctors opening 50 separate connections to the hospital database at the same time—it would crash the server.
The Singleton ensures ClinixTech uses one central, shared database connection.

Exam Keywords:
Private constructor, Global access, Single instance

B. Factory Method Pattern (Creational)

Meaning:
Provides an interface for creating objects in a parent class, but allows child classes to alter the type of objects that will be created.
It hides the complex creation logic from the user.

ClinixTech Example:
When a user logs in, ClinixTech needs to create a dashboard. Instead of writing messy `if/else` statements everywhere, a `UserFactory` handles it.
If the user is a Doctor, the Factory creates a `DoctorDashboard`.
If a Nurse, it creates a `NurseDashboard`.

Exam Keywords:
Decouples creation, Runtime selection, Extensibility

C. Abstract Factory Pattern (Creational)

Meaning:
Lets you produce *families* of related objects without specifying their exact classes.

ClinixTech Example:
UI generation. Suppose ClinixTech runs on both Windows and Mac. The Abstract Factory creates a "Family" of Windows UI components (Windows Button, Windows Menu) or a "Family" of Mac UI components.
This ensures a Mac button never accidentally shows up on a Windows screen.

Exam Keywords:
Families of related objects, Cross-platform compatibility

D. Observer Pattern (Behavioral)

Meaning:
Defines a "one-to-many" subscription mechanism.
When the main object (Subject) changes its state, all subscribed objects (Observers) are automatically notified and updated.

ClinixTech Example:
The **Patient Vitals System**.
The patient's heart rate monitor is the *Subject*.
The `DoctorPhoneApp`, the `NurseStationScreen`, and the `ICU_AlarmSystem` are the *Observers*.
If the patient's heart rate drops, the Subject instantly notifies all three Observers to update their screens and sound alarms.

Exam Keywords:
Subject and Observer, Automatic notifications, Event systems

E. Strategy Pattern (Behavioral)

Meaning:
Lets you define a family of algorithms, put each of them into a separate class, and make their objects interchangeable at runtime.

ClinixTech Example:
The **Patient Billing System**.
A patient needs to pay their bill. The system uses a `PaymentStrategy`.
At runtime, the receptionist can dynamically swap the strategy between `InsurancePayment`, `CreditCardPayment`, or `CashPayment` without changing the core billing code at all.

Exam Keywords:
Interchangeable algorithms, Selected dynamically, Loose coupling

---

4 Recall & Revision Cheat Sheet

- **Singleton:** "There can only be ONE." (Database config)
- **Factory:** "The Object Creator." (Making different User profiles)

- **Abstract Factory:** “The Family Creator.” (Windows vs. Mac UI)
- **Observer:** “The Subscriber.” (YouTube notifications / Vitals alarm)
- **Strategy:** “The Swap-out.” (Swapping payment methods on the fly)

💡 Exam-Oriented Tip

If a past paper asks:
“A system has many complex conditional statements (if/else) to calculate discounts. How do we fix this?”

Your Answer:
Use the **Strategy Pattern**.
It removes the rigid `if/else` logic by turning each discount calculation into its own separate, interchangeable class.

1️⃣ Design Patterns: The Hidden Theory Points

Your PDF mentions a few extra theoretical reasons why we use design patterns:

- **Encapsulation of Change:** Patterns lock away the parts of the code that change often so they don’t break the rest of the system.
- **Promotes SOLID Principles:**
    - Strategy Pattern promotes the **Open/Closed Principle (OCP)** because you can add new strategies without editing old code.
    - Patterns promote the **Dependency Inversion Principle (DIP)** by making high-level modules depend on abstractions (interfaces) rather than low-level concrete classes.

2️⃣ Singleton Pattern (Deep Dive)

Key Components:

- A Private Constructor (stops `new` objects from being made)
- A Static Method (e.g., `get_instance()`) for global access

Advantages:

- Saves memory (only one instance is ever created).
- Strictly controls access to shared resources.

Disadvantages (Exam Favorite):

- Breaks the Single Responsibility Principle (SRP).
- Hard to Unit Test due to global state.

Variations:

- Lazy Initialization
- Eager Initialization
- Thread-Safe Singleton

---

3 Factory Pattern (Deep Dive)

Key Components:

1. Creator / Factory
2. Product Interface
3. Concrete Products

Advantages:

- Loose coupling
- Promotes SRP

Disadvantages:

- Overhead due to many conditionals
- Too complex for small apps

Variations:

- Simple Factory
- Factory Method
- Abstract Factory

---

4 Observer Pattern (Deep Dive)

Key Components:

- Subject
- Observers

Advantages:

- Dynamic subscription
- Scalable for event-driven systems

Disadvantages (Exam Favorite):

- Memory leaks
- Broadcast overhead

Variations:

- Push Model
- Pull Model

---

5 $trategy Pattern (Deep Dive)

Key Components:

- Context
- Strategy Interface
- Concrete Strategies

Advantages:

- Eliminates `if/else` logic
- Runtime flexibility

Disadvantages:

- Client must understand strategies
- Class explosion

Variations:

- Strategy caching
- Lambda expressions
